Component-Level Design

Minsoo Ryu

Hanyang University

Four Design Areas

- **Architecture design**
  - Defines the relationship between major structural elements of the software
- **Component-level design**
  - Transforms structural elements of the software architecture into a procedural description of software components
- **Data/class design**
  - Transforms analysis models into design realizations and the requisite data structures required to implement the software
- **Interface design**
  - Describes how the software communicates with systems that interoperate within it, and with humans who use it

Contents

1. Component-Level Design
2. Fundamental Design Concepts
3. Object-Oriented Design Concepts

Three Design Quality Characteristics

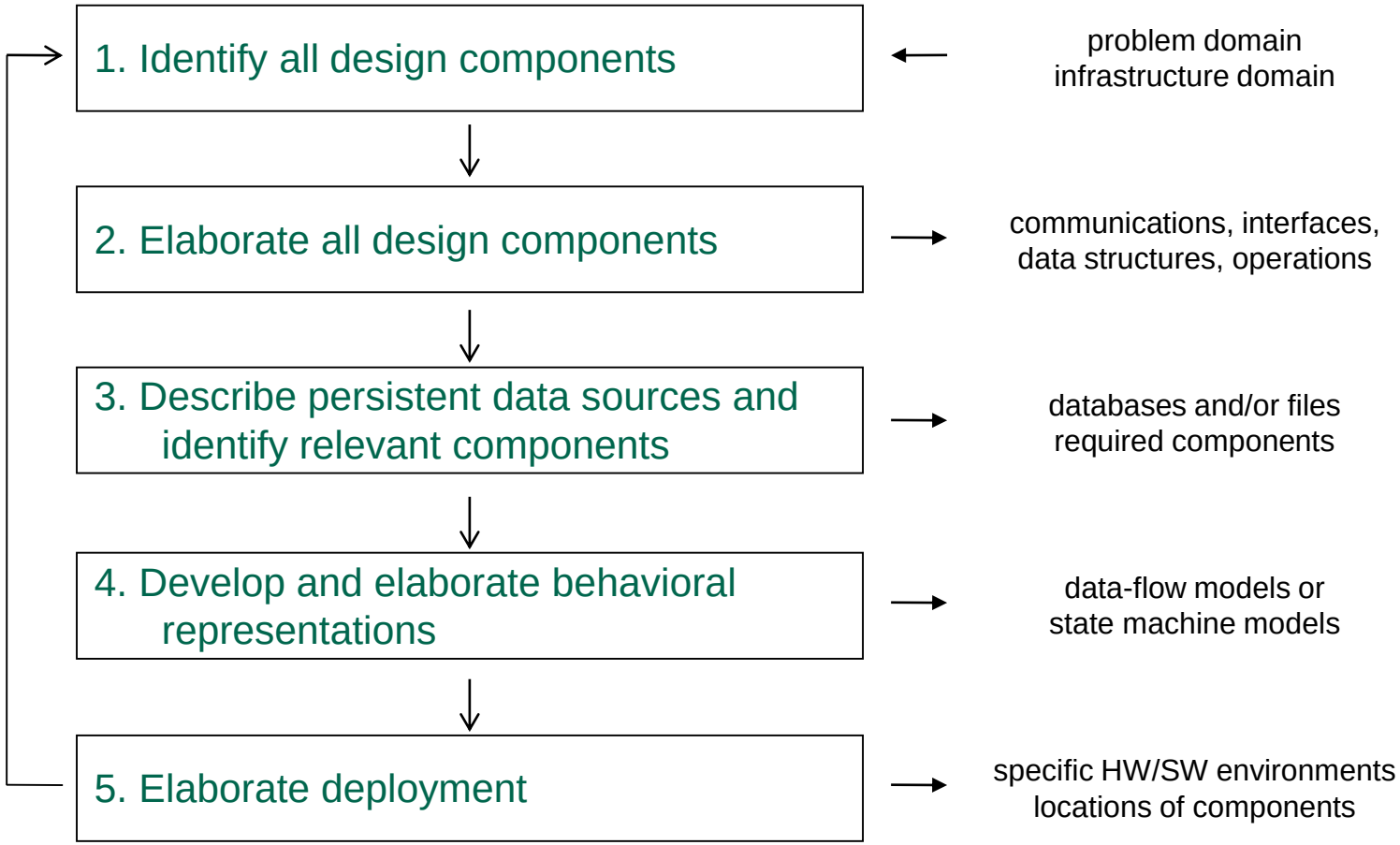
- The design must implement all of the explicit requirements contained in the analysis model, and it must accommodate all of the implicit requirements desired by the customer
- The design must be a readable, understandable guide for those who generate code and for those who test and subsequently support the software
- The design should provide a complete picture of the software, addressing the data, functional, and behavioral domains from an implementation perspective

Component-Level Design

- Component-level design occurs after the first iteration of architectural design has been completed
  - The intent is to define the data structures, processing logic, interface characteristics, and communication mechanisms allocated to each software component

- A component is a functional element of a program
  - A conventional component is also called a "module"
  - OMG Unified Modeling Language Specification defines a component as "a modular, deployable, and replaceable part of a system that encapsulates implementation and exposes a set of interfaces"

Example Flow: Component-Level Design



The order of design steps can vary and some step can be omitted

Step 1: Identify All Design Components

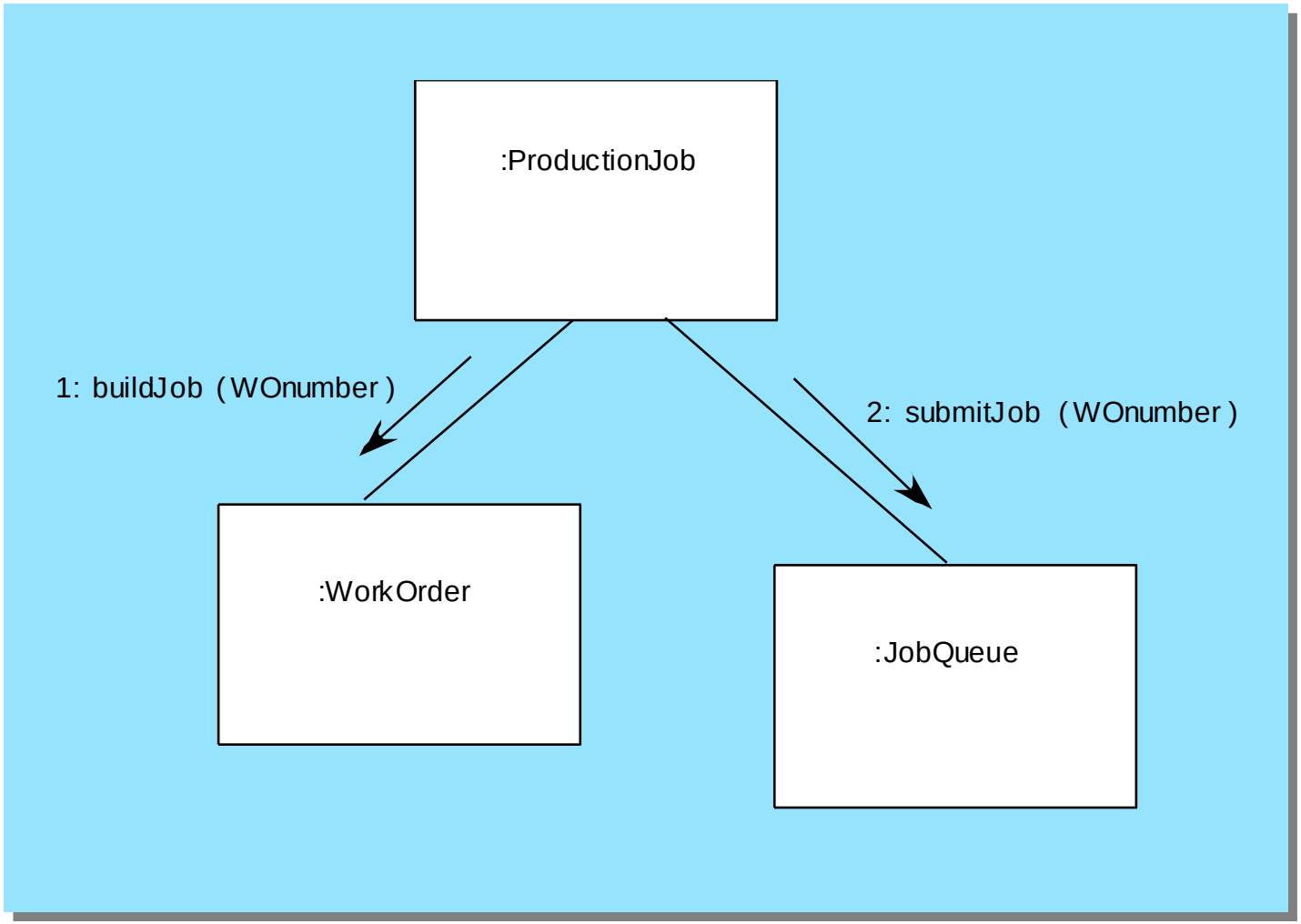
- **Step 1.1**
  - **Identify design components from the problem domain**
  - **Use architectural components and refine them**

- **Step 1.2**
  - **Identify design components from the infrastructure domain**
  - **GUI-related components, OS-related components, etc**

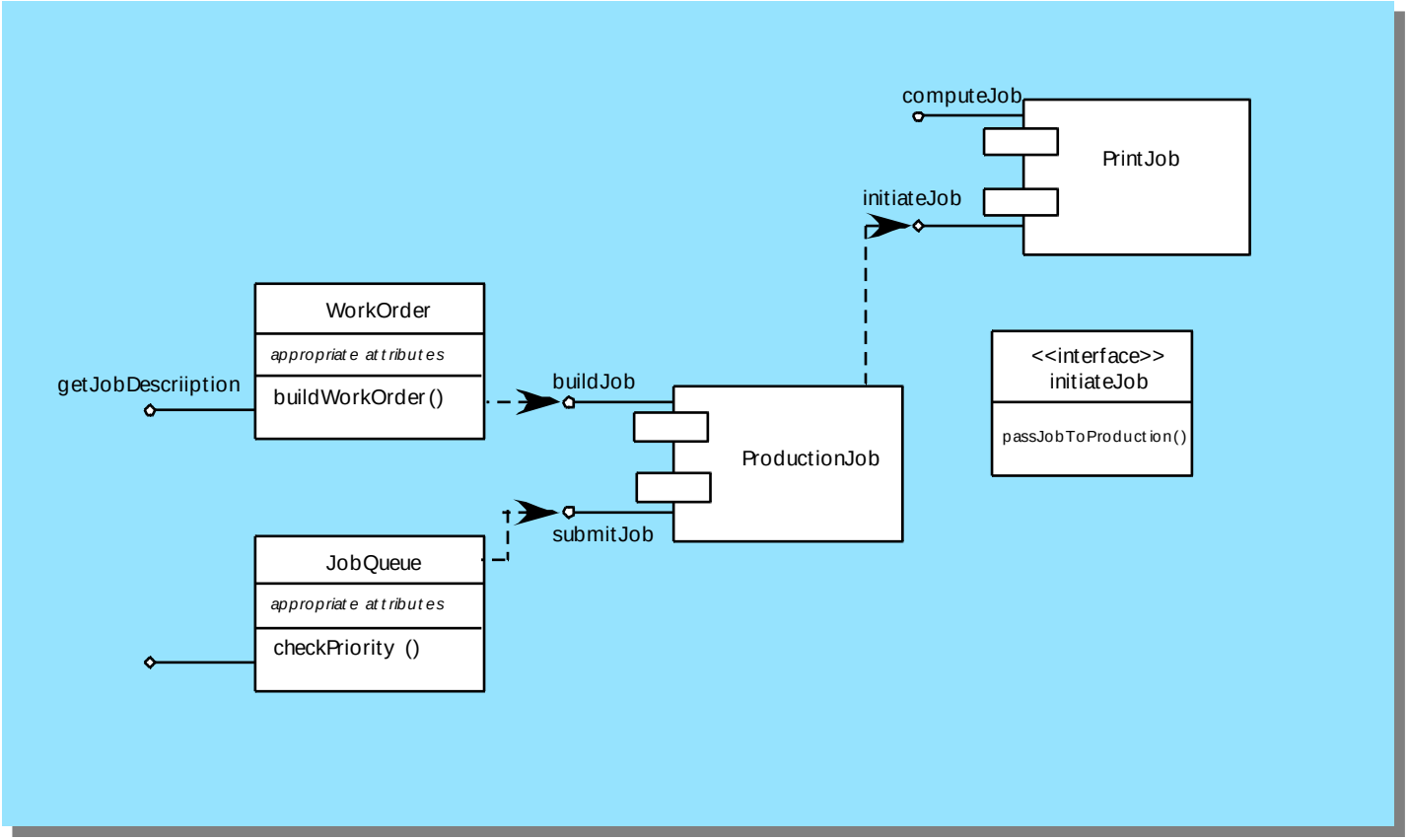
Step 2: Elaborate Design Components

- Step 2.1
  - Specify messages that are passed between design components (message name, content, and properties)
- Step 2.2
  - Identify interfaces for each component (interface name and properties)
- Step 2.3
  - Identify data structures and data types (variables and complex data structures)
- Step 2.4
  - Describe processing flow within each operation (flow chart or UML activity diagram)
  - Specify the parameters and their types for each operation

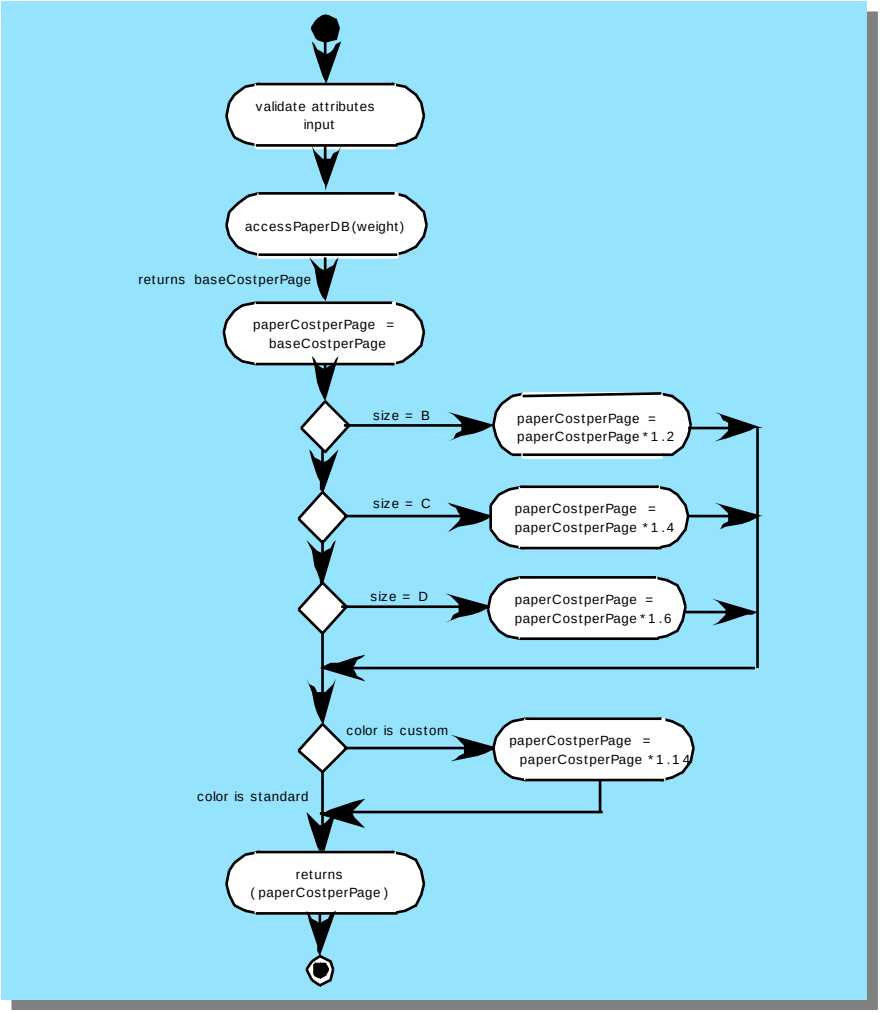
Messages


:ProductionJob
1: buildJob (WOnumber)
2: submitJob (WOnumber)
:WorkOrder
:JobQueue

Interfaces


computeJob
PrintJob
initiateJob
WorkOrder
appropriate attributes
buildWorkOrder()
getJobDescriiption
buildJob
ProductionJob
<<interface>>
initiateJob
passJobToProduction()
submitJob
JobQueue
appropriate attributes
checkPriority ()

Processing Flow



Step 3: Describe Persistent Data Sources and Identify Relevant Components

- In most cases, persistent data stores are initially specified as part of architectural design

- Step 3.1
  - Provide additional detail about the structure and organization of the persistent data stores

- Step 3.2
  - Identify design components that are required to manage them

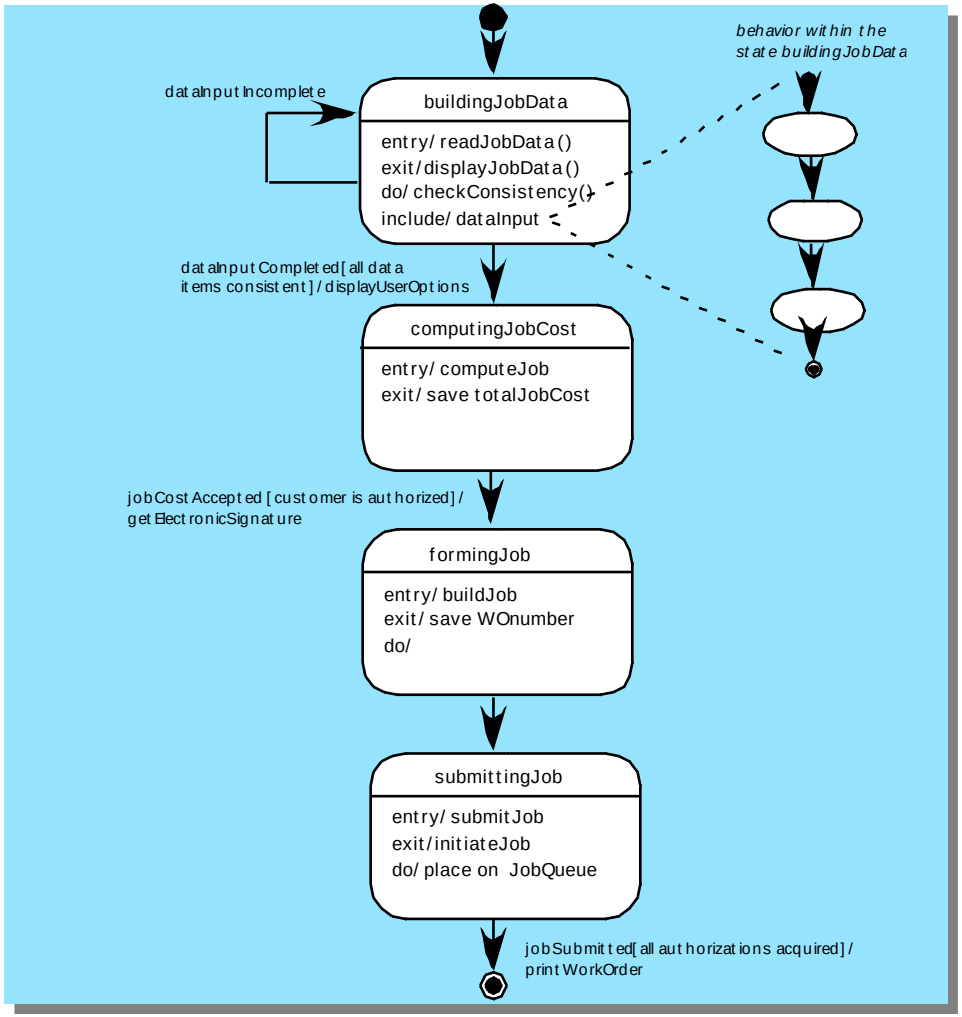
Step 4: Develop and Elaborate Behavioral Representations

- **Use flow charts, data-flow models, state machine models to represent the behavior of entire system and/or of certain component**

- **Step 4.1**
  - **Identify and draw design elements for the overall system**
    - In the case of state machine model: states, events, transitions, and actions

- **Step 4.2**
  - **Identify and draw design elements for individual component**

State Machine Diagram



Step 5: Elaborate Deployment

- **Step 5.1**
  - **Identify the computing environment that will house design components**
- **Step 5.2**
  - **Specify the location of each component**
- **Step 5.3**
  - **Describe specific hardware and/or software environments**

Contents

1. Component-Level Design
2. Fundamental Design Concepts
3. Object-Oriented Design Concepts

Fundamental Design Concepts

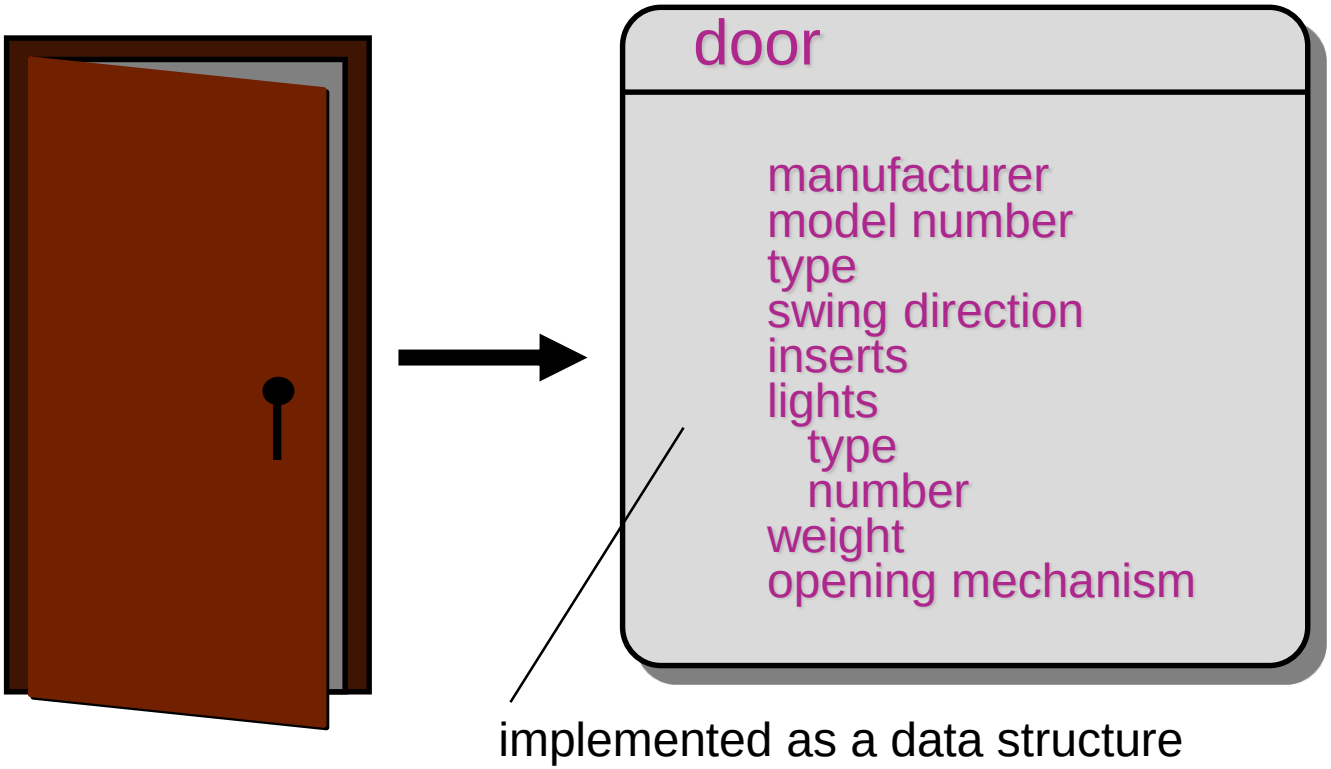
- ❒ Abstraction
- ❒ Modularity
- ❒ Refinement

Abstraction

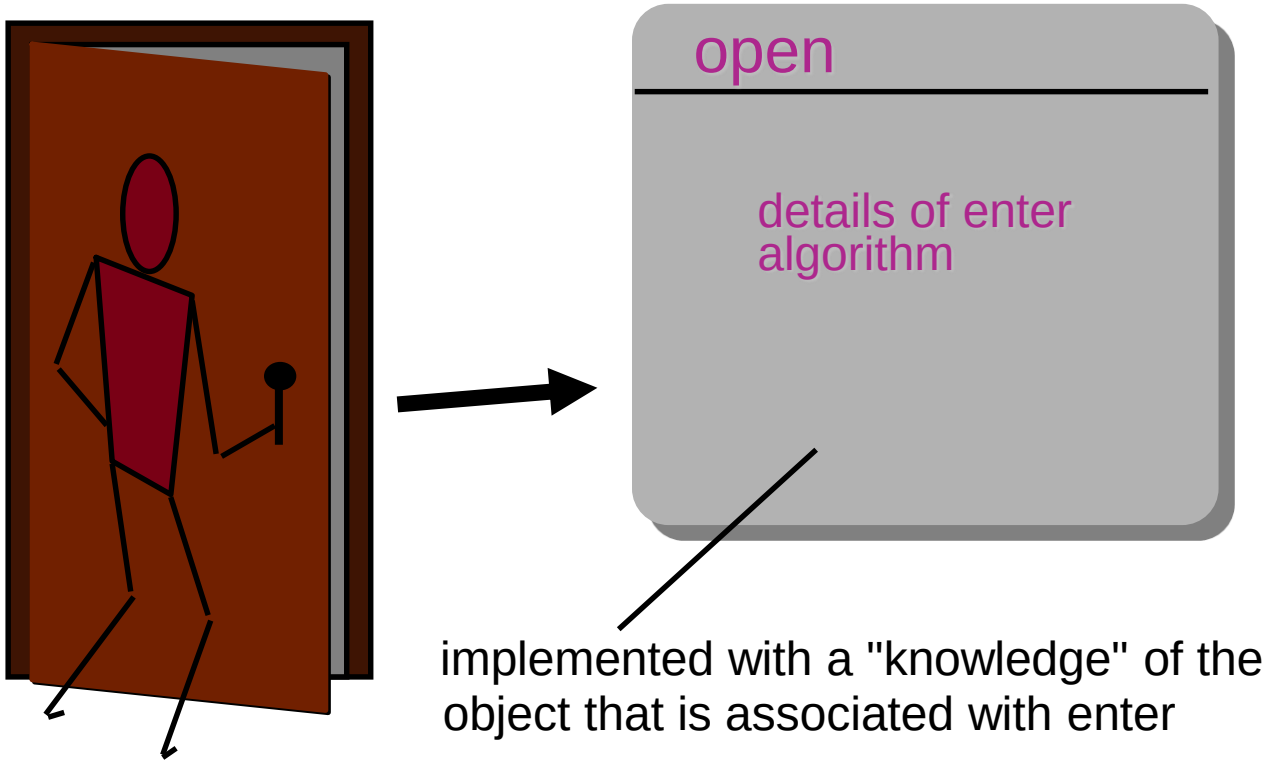
- "Abstraction is one of the fundamental ways that we as humans cope with complexity," by Grady Booch

- Two types of abstraction
  - Data abstraction
  - Procedural abstraction

Data Abstraction



Procedural Abstraction



Modularity

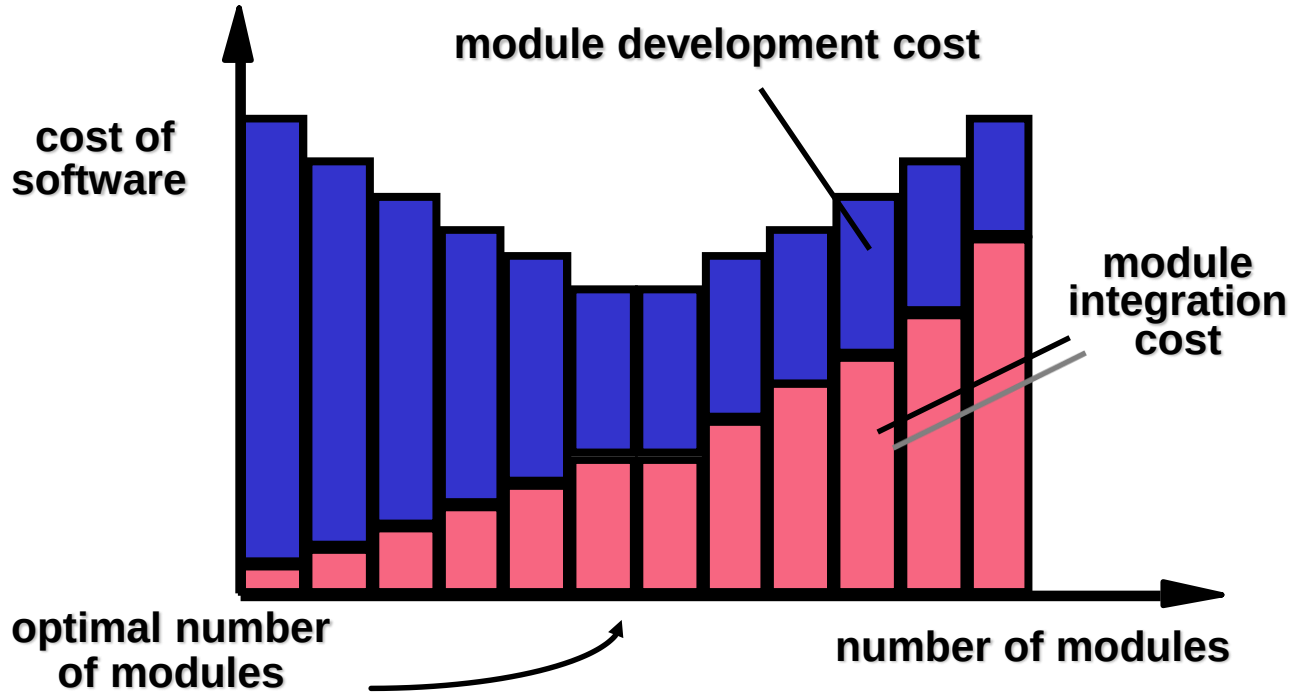
- "Modularity is the single attribute of software that allows a program to be intellectually manageable," by G. Meyers
  - Software can be divided into separately named and addressable components, often called modules, that are integrated to satisfy problem requirements

- Consider two problems, p1 and p2
  - If the perceived complexity of two problems when they are combined is often greater than the sum of the perceived complexity when each is taken separately
  - This leads to a "divide and conquer" strategy

- Two aspects of modularity
  - Information hiding
  - Functional independence

Modularity

What is the "right" number of modules for a specific software design?

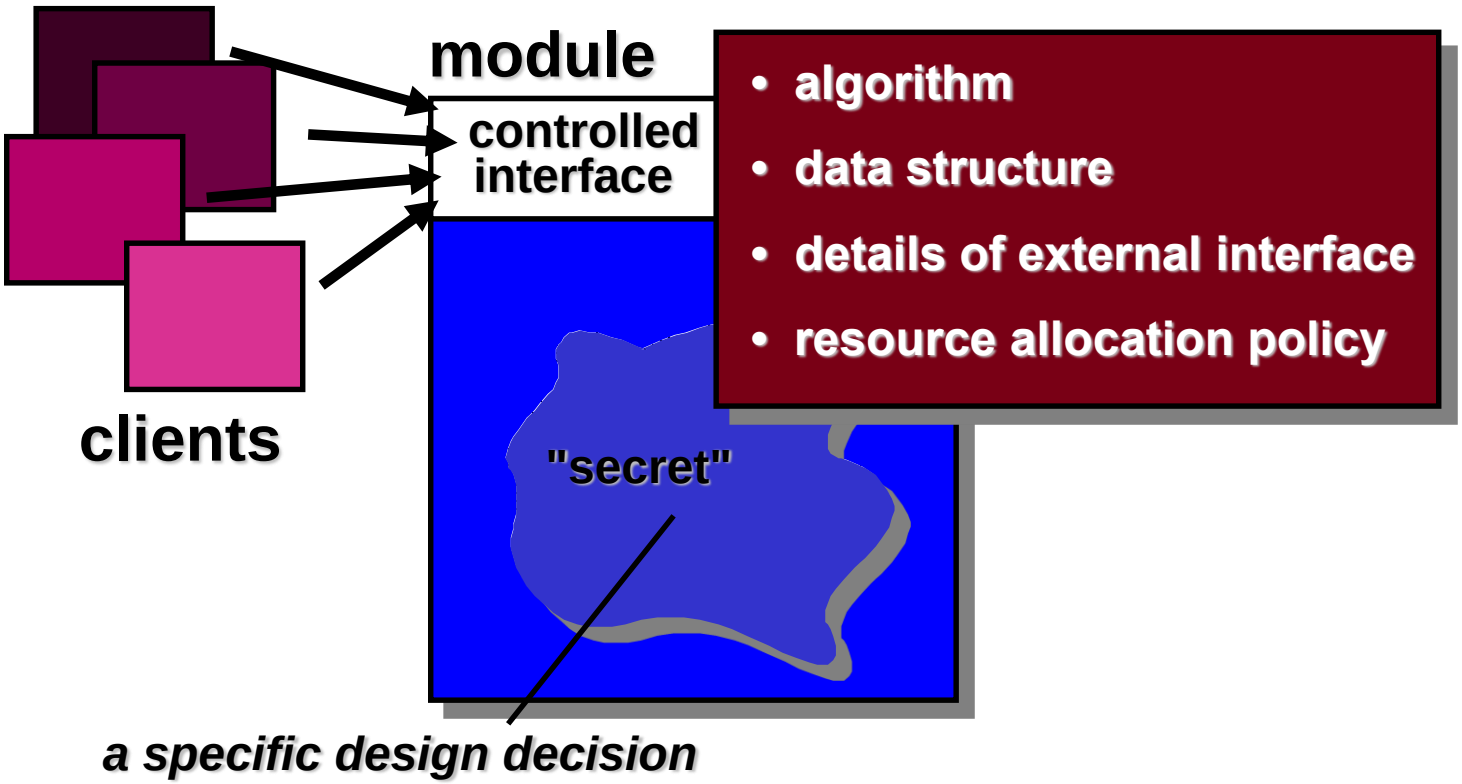


Information Hiding and Encapsulation

- Modules should be specified and designed so that information (algorithms and data) contained within a module is inaccessible to other modules that have no need for such information
  - It is based on the "need-to-know" principle
  - Information hiding shifts the code's dependency on an uncertain implementation (design decision) onto a well-defined public interface

- The term encapsulation is often used interchangeably with information hiding
  - The internal mechanisms of the component can be improved without impact on other components, or the component can be replaced with a different one that supports the same public interface
  - Encapsulation also protects the integrity of the component, by preventing users from setting the internal data of the component into an invalid or inconsistent state
  - Another benefit of encapsulation is that it reduces system complexity and thus increases robustness, by limiting the interdependencies between software components

Information Hiding and Encapsulation



Functional Independence

- **The functional independence is a direct outgrowth of modularity and the concept of abstraction**

- **Independence is assessed using two criteria**
  - **Cohesion is an indication of the relative functional strength of a module**
    - A cohesive module consists of parts that fit together well and form a united whole
    - A cohesive module should (ideally) do just one thing
  - **Coupling is an indication of the relative interdependence among modules**

Cohesion

- Cohesion implies that a component or class encapsulates only attributes and operations that are closely related to one another and to the class or component itself

- Functional (exhibited primarily by operations)
  - This type of cohesion occurs when a module performs one and only one computation
- Communicational
  - All operations that access the same data are defined within one class
- Temporal
  - Operations that are performed to reflect a specific behavior or state, e.g., an operation performed at start-up or all operations performed when an error is detected

Coupling

- Coupling is a qualitative measure of the degree to which classes are connected to one another
  - As classes become more interdependent, coupling increases
  - An important objective in component-level design is to keep coupling as low as possible

- Content coupling
  - Occurs when one component modifies data that is internal to another component (This violates information hiding)
- Common coupling
  - Occurs when a number of components all make use of a global variable
  - Although this sometimes necessary, common coupling can lead to uncontrolled error propagation and unforeseen side effects when changes are made
- Control coupling
  - Occurs when operation A() invokes operation B() and passes a control flag to B

Refinement

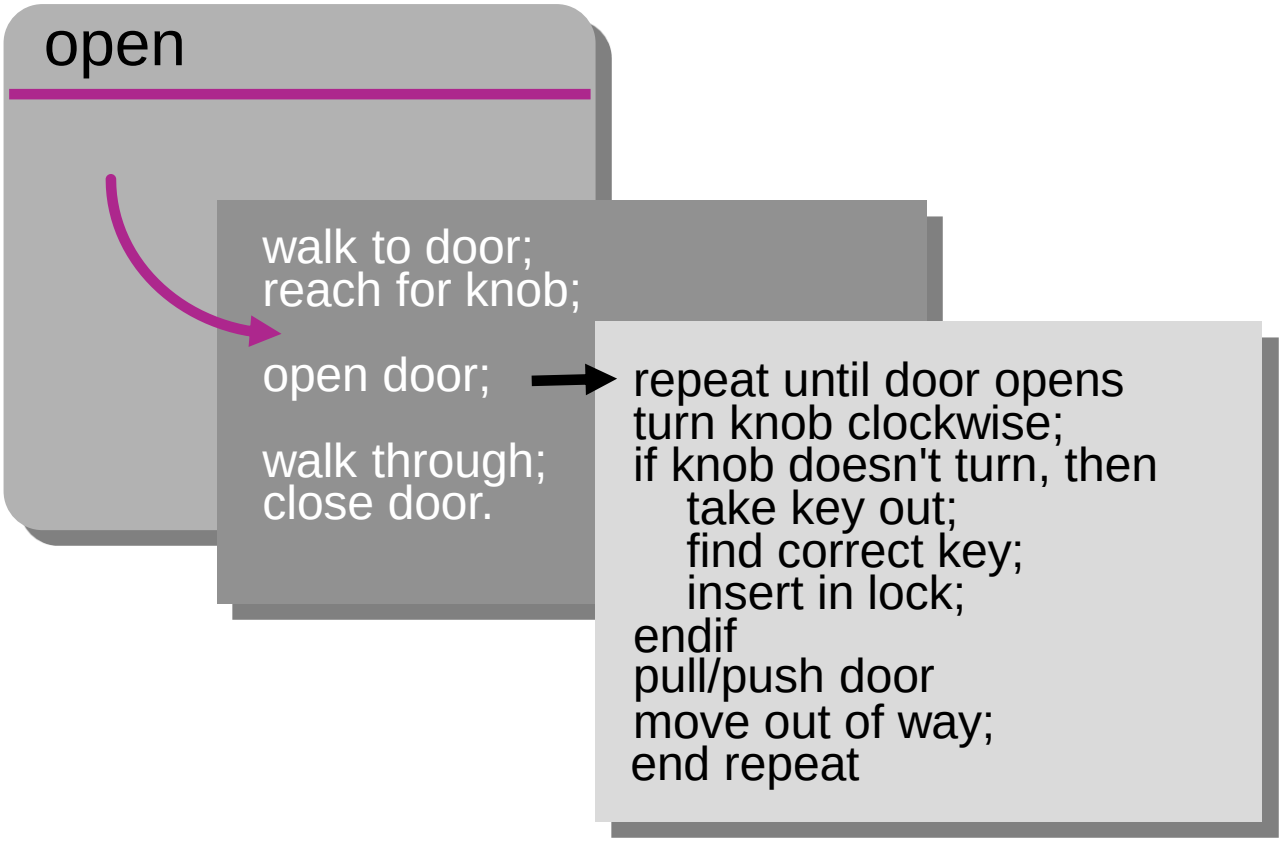
- Step-wise refinement is a top-down design strategy
  - Stepwise refinement is the idea that software is developed by moving through the levels of abstraction, beginning at higher levels and, incrementally refining the software through each level of abstraction, providing more detail at each increment

- The term stepwise refinement was used first in the paper titled “Program Development by Stepwise Refinement” by Niklaus Wirth, in the Communications of the ACM, Vol. 14 (4), 1971, pp. 221-227
  - Niklaus Wirth is the author of the programming language Pascal and other major contributions to software design and software engineering

Stepwise Refinement

- Stepwise refinement can be carried out by:
  - Refining a difficult problem into two steps
  - Breaking a high-level activity into a sequence of sub-activities
  - Breaking a task into a sequence of tasks
  - Introducing variables to capture system states and attributes
  - Structuring a loop to reflect the structure of the data it processes

Stepwise Refinement



Stepwise Refinement

- **Stepwise refinement may require:**
  - **Separation of concerns**
    - A key point is to focus on one thing at a time and handle it properly (coined by Edsger Dijkstra)
  - **The mañana principle**
    - Seeing the need for a method or service, stubbing it in and proceed to design using it, and then later (tomorrow, mañana) designing and/or implementing it

- **Abstraction and refinement are complementary concepts**
  - **Abstraction enables a designer to specify procedure and data and yet suppress low-level details**
  - **Refinement helps the designer to reveal low-level details as design progresses**


Q & A